

SISTEMAS OPERATIVOS

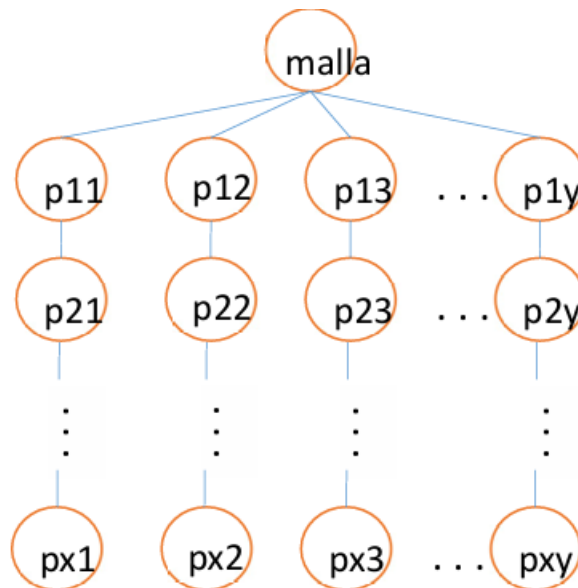
PRÁCTICA 1

Gestión de procesos y archivos.
Comunicación entre procesos: tuberías
y memoria compartida

Ejercicio 1:

1.1. Enunciado del ejercicio

a) Realiza un programa llamado `mallá.c` que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "`pstree -c`".



1.2. Funciones

1.2.1. Variables globales

Para gestionar la dimensión de la matriz de procesos a lo largo de las distintas funciones del programa, se emplean dos variables globales de tipo entero:

- **x**: variable de tipo integer que almacena el número de filas.
- **y**: variable de tipo integer que almacena el número de columnas.

```
mallá.c
```

```
int x = 0; int y = 0;
```

1.2.2. Función *main()*

La función **main()** actúa como punto de entrada del programa. En primer lugar, delega la lectura y validación de los argumentos de la línea de comandos invocando a **process_args()**. A continuación, llama a la función **crear_malla()**, que se encargará de coordinar toda la creación del árbol de procesos. Finalmente, retorna 0 indicando la finalización con éxito del proceso principal.

```
malla.c
```

```
int main(int argc, char *argv[]) {
    process_args(argc, argv);
    crear_malla();
    return 0;
}
```

1.2.3. Función *process_args()*

La función **process_args()** se encarga de validar los argumentos introducidos por el usuario desde la línea de comandos:

- Comprueba si el número de argumentos pasados (= argc) es inferior a 3 (no se han especificado los valores de x e y). En caso de error, el programa aborta inmediatamente con la llamada **exit(EXIT_FAILURE)**.
- Si los parámetros son válidos, utiliza la función **atoi()** para convertir los argumentos en formato texto (argv[1] y argv[2]) a enteros y almacenarlos en las variables globales x e y.

```
malla.c
```

```
void process_args(int argc, char *argv[]) {
    if (argc < 3) {
        exit(EXIT_FAILURE);
    }
    x = atoi(argv[1]);
    y = atoi(argv[2]);
}
```

1.2.4. Función *crear_malla()*

La función **crear_malla()** genera la dimensión horizontal del árbol a partir del proceso raíz malla:

- Mediante un bucle **for** que itera 'y' veces, el proceso principal realiza una llamada a **fork()** en cada iteración.

- Si **fork()** devuelve 0, nos encontramos en el proceso hijo recién creado. Este hijo invoca a **crear_columna()** para generar su rama vertical.
- En el proceso padre, **fork()** devuelve el PID del hijo (distinto de 0), por lo que continúa iterando en el bucle para crear las columnas restantes.
- Tras crear las 'y' columnas, el proceso principal entra en un bucle while. Esto garantiza que el proceso padre permanezca en espera hasta que todos sus hijos directos terminen, asegurando así que la estructura completa del árbol permanezca visible en el sistema mientras se comprueba.

mallla.c

```
void crear_malla(void) {
    for (int i = 0; i < y; i++) {
        if (fork() == 0) {
            crear_columna();
        }
    }
    while (wait(NULL) > 0);
}
```

1.2.5. Función **crear_columna()**

La función **crear_columna()** es la encargada de construir la cadena vertical descendente de procesos hasta alcanzar los 'x' niveles de profundidad:

- Utiliza un bucle for que va desde j = 1 hasta x - 1.
- En cada nivel, el proceso ejecuta la llamada **fork()**.
- Si **fork()** retorna un valor distinto de 0 (nos encontramos en el proceso padre de ese nivel específico), este esperará a la terminación de su hijo directo mediante **wait(NULL)** y posteriormente finalizará de manera limpia mediante **exit(EXIT_SUCCESS)**. Con esta estructura se consigue una destrucción ordenada en cadena desde las hojas hacia la raíz.
- Si **fork()** retorna 0 (proceso hijo), se continúa el bucle para crear el siguiente nivel hacia abajo.

mallla.c

```
void crear_columna(void) {
    for (int j = 1; j < x; j++) {
        if (fork() != 0) {
```

```
        wait(NULL);  
        exit(EXIT_SUCCESS);  
    }  
}  
ejecutar_hoja();  
}
```

1.2.6. Función *ejecutar_hoja()*

La función **ejecutar_hoja()** mantiene activos los procesos hojas durante un intervalo de tiempo para poder inspeccionar la jerarquía de procesos creada en el sistema operativo:

- Asocia la señal **SIGALRM** a la función **manejador_alarma** mediante la llamada **signal()**.
- Configura una alarma de 10 segundos con **alarm(10)**.
- Detiene la ejecución del proceso hoja en espera pasiva mediante **pause()**.
Transcurridos los 10 segundos, la señal **SIGALRM** despierta al proceso, liberando la llamada **pause()** y permitiéndole finalizar de forma ordenada con **exit(EXIT_SUCCESS)**.

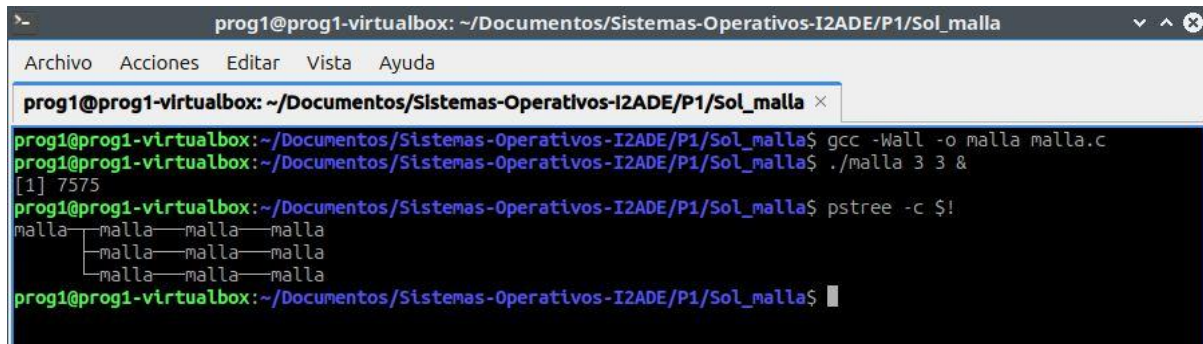
```
mall.c  
void ejecutar_hoja(void) {  
    signal(SIGALRM, manejador_alarma);  
    alarm(10);  
    pause();  
    exit(EXIT_SUCCESS);  
}
```

1.2.7. Función *manejador_alarma()*

La función **manejador_alarma()** sirve como rutina de tratamiento de la señal **SIGALRM**. Al dejarse intencionadamente vacía, su único objetivo es capturar la señal para evitar la muerte por defecto del proceso, desbloqueando así el **pause()** dentro de **ejecutar_hoja()**.

```
mall.c  
void manejador_alarma(int sig) {  
  
}
```

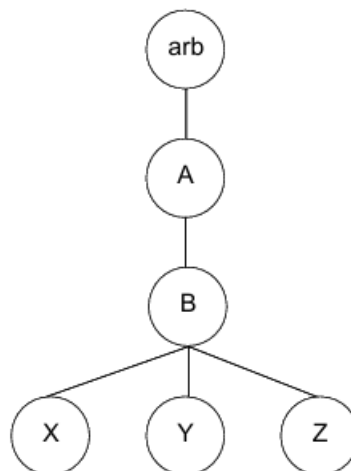
1.3. Salida esperada



```
prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla
Archivo Acciones Editar Vista Ayuda
prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla x
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla$ gcc -Wall -o malla malla.c
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla$ ./malla 3 3 &
[1] 7575
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla$ pstree -c $!
malla--malla--malla--malla
      |malla--malla--malla
      |malla--malla--malla
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_malla$
```

1.1. Enunciado del ejercicio

b) Realiza un programa llamado `ejec.c` que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando “`pstree`”. El proceso Z no puede utilizar el comando “`sleep`”, por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).



1.2. Funciones del código

1.2.1. Variables globales

En este ejercicio se van a usar variables globales para facilitar la tarea. Las variables empleadas son las siguientes:

- `pidEjec`, `pidA`, `pidB`, `pidX`, `pidY` y `pidZ`, todas de tipo `pid_t`, son variables que almacenarán los PIDs de los procesos correspondientes a lo largo del ejercicio.
- `secs`, de tipo `integer`, es la variable donde guardaremos el tiempo (en segundos) introducido por el usuario para configurar la alarma del proceso Z.

```
ejec.c
```

```
pid_t pidEjec;  
pid_t pidA;  
pid_t pidB;  
pid_t pidX;  
pid_t pidY;  
pid_t pidZ;  
  
int secs;
```

1.2.2. Función `main()`

La función **`main()`** actúa como el proceso '**Super Padre**' (raíz del árbol). Lo primero que hará será procesar los argumentos invocando a **`processArgs()`**. A continuación, se almacena el PID del proceso actual en la variable global **`pidEjec`** y se imprime su correspondiente mensaje de presentación.

Se prepara al proceso raíz para recibir la señal de finalización armando el manejador **`handler_start_destruction`** para la señal `SIGUSR2` usando la llamada **`signal()`**. Después, llama a **`make_processA()`** para comenzar la creación de la rama descendente. Finalmente, tras la finalización del proceso A, la raíz padre imprime su mensaje anunciando su muerte y retorna 0.

```
ejec.c
```

```
int main(int argc, char *argv[]) {  
    processArgs(argc, argv);  
  
    pidEjec = getpid();  
    printf("Soy el proceso ejec: mi pid es %d\n", pidEjec);  
  
    signal(SIGUSR2, handler_start_destruction);  
  
    make_processA();  
}
```

```
printf("Soy ejec (%d) y muero\n", pidEjec);  
return 0;  
}
```

1.2.3. Función *processArgs()*

La función **processArgs()** comprueba los argumentos pasados por el usuario. Si el número de argumentos introducidos es diferente de 2, entonces se imprime un mensaje de error por la salida estándar de errores (stderr) y el programa aborta. De ser el número correcto, se convierte la cadena de caracteres introducida a entero mediante **atoi()** y se almacena en la variable global **secs**.

```
ejec.c  
void processArgs(int argc, char *argv[]) {  
    if (argc != 2) {  
        fprintf(stderr, "Uso: %s <segundos>\n", argv[0]);  
        exit(EXIT_FAILURE);  
    }  
    secs = atoi(argv[1]);  
}
```

1.2.4. Funciones del proceso A (*make_processA()* y *exec_processA()*)

La función **make_processA()** realiza una llamada a **fork()** cuyo valor de retorno queda almacenado en la variable **pidA**. Si es el proceso padre, el valor devuelto será distinto de 0 y se quedará a la espera con **wait(NULL)**. Si el valor es 0, estamos en el hijo y se invoca **exec_processA()** seguida de un **exit(0)**.

Dentro de **exec_processA()**, se guarda el PID real del proceso en **pidA** empleando **getpid()** y se imprime por pantalla indicando además el PID de su padre (pidEjec). Seguidamente, se arman dos señales:

- **SIGUSR1**, con manejador **handler_A_execTask**, que se encargará de ejecutar el comando cuando Z lo indique.
- **SIGUSR2**, con manejador **handler_A_destroy_propagate**, para propagar la orden de destrucción hacia abajo. Finalmente, se llama a **make_processB()** para continuar haciendo crecer el árbol.

```
ejec.c  
void make_processA(void) {  
    switch (pidA = fork()) {  
        case 0:
```

```
        exec_processA();
        exit(0);
    default:
        wait(NULL);
}

void exec_processA(void) {
    pidA = getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", pidA, pidEjec);

    signal(SIGUSR1, handler_A_execTask);
    signal(SIGUSR2, handler_A_destroy_propagate);

    make_processB();
}
```

1.2.5. Proceso B (*make_processB()* y *exec_processB()*)

La lógica de **make_processB()** es bastante similar a la del proceso anterior: uso de **fork()** y **wait(NULL)** en el caso del padre.

En **exec_processB()**, tras inicializar su **pidB** e imprimir su mensaje de creación informando sobre su jerarquía (padre y abuelo), el proceso prepara el manejador **handler_B_destroy_propagate** para capturar la señal **SIGUSR2**. Su labor central consiste en invocar secuencialmente a **make_processX()**, **make_processY()** y **make_processZ()** para generar las tres hojas del árbol. Tras ello, el proceso B entra en un bucle infinito de espera pasiva empleando **pause()** hasta que llegue la señal de destrucción.

```
ejec.c
void make_processB(void) {
    switch (pidB = fork()) {
        case 0:
            exec_processB();
            exit(0);
        default:
            wait(NULL);
    }
}

void exec_processB(void) {
    pidB = getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n", pidB, pidA, pidEjec);
}
```

```
    signal(SIGUSR2, handler_B_destroy_propagate);

    make_processX();
    make_processY();
    make_processZ();

    while (1) {
        pause();
    }
}
```

1.2.6. Procesos X() e Y()

Las funciones relativas a X (**make_processX**, **exec_processX**) e Y (**make_processY**, **exec_processY**) actúan como hojas inactivas. Tras ser creadas mediante su respectivo **fork()**, cada una guarda su **PID**, imprime el mensaje detallando su árbol genealógico completo y asocia la señal **SIGUSR2** a sus respectivos manejadores de muerte (**handler_X_destroyLeaf** y **handler_Y_destroyLeaf**). Ambos procesos quedan retenidos en un bucle con **pause()** a la espera del final de la ejecución.

1.2.7. Proceso Z()

El proceso Z comparte la estructura base de X e Y, pero posee una importante labor temporal en el ejercicio. Dentro de **exec_processZ()**, se arma **SIGALRM** con la función manejadora **handler_Z_alarm** (que enviará la señal a A). Para activar esta lógica transcurrido el tiempo, se invoca a la llamada **alarm(secs)**, pasándole la cantidad de segundos introducida por el usuario al inicio. Después, entra en el bucle infinito con **pause()** aguardando a que la alarma finalice y a que, posteriormente, el proceso B le envíe la señal de destrucción.

ejec.c

```
void exec_processZ(void) {
    pidZ = getpid();
    printf("Soy el proceso Z: mi pid es %d. Mi padre es %d.
    Mi abuelo es %d. Mi bisabuelo es %d\n",
           pidZ, pidB, pidA, pidEjec);

    signal(SIGALRM, handler_Z_alarm);
    signal(SIGUSR2, handler_Z_destroyLeaf);

    alarm(secs);

    while (1) {
        pause();
    }
}
```

1.2.8. Manejadores de señales y propagación de destrucción

El ejercicio exige una ejecución de un comando y una destrucción controlada (padres esperando a hijos). Esto se resuelve interconectando las señales:

1. **Activación de Tarea (handler_Z_alarm y handler_A_execTask):** Cuando expira la alarma en Z, este manejador envía **SIGUSR1** al proceso A mediante **kill(pidA, SIGUSR1)**. Al recibirla, A ejecuta **handler_A_execTask**, donde hace un nuevo **fork()** que en su proceso hijo invoca **execlp("pstree", "pstree", (char *) NULL)** para mostrar el árbol. El padre A usa **wait(NULL)** y luego envía **SIGUSR2** a la raíz ejec para notificarle que ha terminado.
2. **Propagación de Destrucción (handler_*_destroy_propagate):** El proceso raíz captura ese **SIGUSR2** y devuelve **SIGUSR2** a A (**kill(pidA, SIGUSR2)**).
 - a. **Manejador en A:** Notifica a B (**kill(pidB, SIGUSR2)**) y hace **wait(NULL)** antes de imprimir su mensaje de muerte y hacer **exit(0)**.
 - b. **Manejador en B:** Manda **SIGUSR2** secuencialmente a Z, a Y y a X, realizando un **wait(NULL)** por cada hijo destruido para garantizar que ninguno quede huérfano, imprimiendo su mensaje de muerte al final.
 - c. **Manejadores Hojas:** Únicamente imprimen su PID e invocan **exit(0)**.

`ejec.c`

```
void handler_A_execTask(int s) {
    pid_t pid;
    (void)s;

    pid = fork();
    if (pid == 0) {
        execlp("pstree", "pstree", (char *) NULL);
        exit(EXIT_FAILURE);
    }
    wait(NULL);
    kill(pidEjec, SIGUSR2);
}

void handler_A_destroy_propagate(int s) {
    (void)s;
    kill(pidB, SIGUSR2);
    wait(NULL);

    printf("Soy A (%d) y muero\n", pidA);
    exit(0);
}
```

1.3. Salida esperada

```

prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec$ gcc Wall -o ejec_ejec.c
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec$ ./ejec 15
Soy el proceso ejec: mi pid es 1620
Soy el proceso A: mi pid es 1621, mi padre es 1620
Soy el proceso B: mi pid es 1622, mi padre es 1621, mi abuelo es 1620
Soy el proceso X: mi pid es 1623, mi padre es 1622, mi abuelo es 1621, mi bisabuelo es 1620
Soy el proceso Y: mi pid es 1624, mi padre es 1622, mi abuelo es 1621, mi bisabuelo es 1620
Soy el proceso Z: mi pid es 1625, mi padre es 1622, mi abuelo es 1621, mi bisabuelo es 1620
systemd--NodeManager--3*[{NodeManager}]
--NetworkManager--3*[{NetworkManager}]
--accounts-daemon--3*[{accounts-daemon}]
--anacron
--avahi-daemon--avahi-daemon
--cron
--cups-browsed--3*[{cups-browsed}]
--cupsd--4*[{dbus}]
--dbus-daemon
--2*[{kerneloops}]
--polkitd--3*[{polkitd}]
--rsyslogd--3*[{rsyslogd}]
--rtkit-daemon--2*[{rtkit-daemon}]
--sdm--3*[{Korq}]
--sddm-helper--lxqt-session--3*[{agent}]
--aplet-py
--at-spi-bus-launcher--dbus-daemon
--at-spi2-registrars--4*[{at-spi2-registrars}]
--blueman-applet--4*[{blueman-applet}]
--lubuntu-update--2*[{lubuntu-update}]
--lxqt-globalkeys--3*[{lxqt-globalkeys}]
--lxqt-notificationd--2*[{lxqt-notificationd}]
--lxqt-panel--4*[{lxqt-panel}]
--lxqt-policykit--3*[{lxqt-policykit}]
--lxqt-powermanag--3*[{lxqt-powermanag}]
--lxqt-runner--2*[{lxqt-runner}]
--nm-tray--2*[{nm-tray}]
--openbox--2*[{openbox}]
--pcmanfm-qt--3*[{pcmanfm-qt}]
--picon
--qlipper--2*[{qlipper}]
--qterminal--bash--ejec--ejec--ejec--3*[{ejec}]
--2*[{qterminal}]
--ssh-agent
--xscreensaver--xscreensaver-sy
--3*[{lxqt-session}]
--[sddm]
--snapd--7*[{snapd}]
--switcheroo-control--3*[{switcheroo-control}]
--systemd--(sd-pan)
--dbus-daemon
--gnome-keyring-d--4*[{gnome-keyring-d}]
--gvfs-afc-volume--4*[{gvfs-afc-volume}]
--gvfs-goa-volume--3*[{gvfs-goa-volume}]
--gvfs-gphoto2-vol--3*[{gvfs-gphoto2-vol}]
--gvfs-mtp-volume--3*[{gvfs-mtp-volume}]
--gvfs-udisks2-vol--4*[{gvfs-udisks2-vol}]
--gvfsd
--3*[{gvfsd}]
--gvfsd-fuse--7*[{gvfsd-fuse}]
--kscreen-backend--2*[{kscreen-backend}]
--obex
--2*[{pipewire}]
--pipewire-pulse--2*[{pipewire-pulse}]
--wireplumber--3*[{wireplumber}]
--systemd-journal
--systemd-logind
--systemd-resolve
--systemd-timesyn--(systemd-timesyn)
--systemd-udevd
--udisksd--3*[{udisksd}]
--unattended-upgr--(unattended-upgr)
--upowerd--3*[{upowerd}]
--wpa_supplicant
Soy Z (1625) y muerto
Soy Y (1624) y muerto
Soy X (1623) y muerto
Soy B (1622) y muerto
Soy A (1621) y muerto
Soy ejec (1620) y muerto
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec$

```

```

prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec$
--at-spi-bus-launcher--4*[{at-spi-bus-launcher}]
--at-spi2-registrars--3*[{at-spi2-registrars}]
--blueman-applet--4*[{blueman-applet}]
--lubuntu-update--2*[{lubuntu-update}]
--lxqt-globalkeys--3*[{lxqt-globalkeys}]
--lxqt-notificationd--2*[{lxqt-notificationd}]
--lxqt-panel--4*[{lxqt-panel}]
--lxqt-policykit--3*[{lxqt-policykit}]
--lxqt-powermanag--3*[{lxqt-powermanag}]
--lxqt-runner--2*[{lxqt-runner}]
--nm-tray--2*[{nm-tray}]
--openbox--2*[{openbox}]
--pcmanfm-qt--3*[{pcmanfm-qt}]
--picon
--qlipper--2*[{qlipper}]
--qterminal--bash--ejec--ejec--ejec--3*[{ejec}]
--2*[{qterminal}]
--ssh-agent
--xscreensaver--xscreensaver-sy
--3*[{lxqt-session}]
--[sddm]
--snapd--7*[{snapd}]
--switcheroo-control--3*[{switcheroo-control}]
--systemd--(sd-pan)
--dbus-daemon
--gnome-keyring-d--4*[{gnome-keyring-d}]
--gvfs-afc-volume--4*[{gvfs-afc-volume}]
--gvfs-goa-volume--3*[{gvfs-goa-volume}]
--gvfs-gphoto2-vol--3*[{gvfs-gphoto2-vol}]
--gvfs-mtp-volume--3*[{gvfs-mtp-volume}]
--gvfs-udisks2-vol--4*[{gvfs-udisks2-vol}]
--gvfsd
--3*[{gvfsd}]
--gvfsd-fuse--7*[{gvfsd-fuse}]
--kscreen-backend--2*[{kscreen-backend}]
--obex
--2*[{pipewire}]
--pipewire-pulse--2*[{pipewire-pulse}]
--wireplumber--3*[{wireplumber}]
--systemd-journal
--systemd-logind
--systemd-resolve
--systemd-timesyn--(systemd-timesyn)
--systemd-udevd
--udisksd--3*[{udisksd}]
--unattended-upgr--(unattended-upgr)
--upowerd--3*[{upowerd}]
--wpa_supplicant
Soy Z (1625) y muerto
Soy Y (1624) y muerto
Soy X (1623) y muerto
Soy B (1622) y muerto
Soy A (1621) y muerto
Soy ejec (1620) y muerto
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_ejec$

```

Ejercicio 2:

2.1. Enunciado del ejercicio

Realizar un programa llamado `hacha.c` que divida un archivo en varios trozos con el mismo nombre y extensión `h00`, `h01`, Se debe tener en cuenta el formato y consideraciones siguientes: \$ `hacha` nombre del archivo a dividir tamaño en bytes de los archivos divididos ¶ Para dividir, el proceso `hacha` (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

2.2. Funciones

2.2.1. Variables globales

Para el siguiente programa, se utilizan las siguientes variables globales:

- **nombreOrigen**: puntero de tipo `char*` que almacena la cadena con el nombre o ruta del archivo de entrada que se va a trocear.
- **tamTrozo**: variable de tipo `integer` que indica el tamaño máximo en bytes especificado por el usuario para cada fragmento.
- **numTrozos**: variable de tipo `integer` que almacena el número total de fragmentos que se generarán tras calcular el tamaño total del fichero.

```
hacha.c
char *nombreOrigen;
int tamTrozo;
int numTrozos;
```

2.2.2. Función `main()`

La función **`main()`** actúa como punto de entrada y coordinador principal:

- En primer lugar, delega la validación de los parámetros recibidos por la consola invocando a **`processArgs()`**.
- Llama a **`calcularTrozos()`**, pasándole el nombre del archivo y el tamaño de trozo, para obtener cuántos fragmentos se deberán crear en total según el tamaño en disco del fichero original.
- Invoca a **`divFich()`**, encargada de gestionar el bucle de creación de tuberías, generación de hijos y lectura/escritura de datos.
- Retorna 0 tras completar el proceso con éxito.

hacha.c

```
int main(int argc, char *argv[]) {
    processArgs(argc, argv);

    numTrozos = calcularTrozos(nombreOrigen, tamTrozo);

    divFich();

    return 0;
}
```

2.2.2. Función *processArgs()*

La función **processArgs()** realiza la lectura y comprobación de los argumentos suministrados en la línea de comandos:

- Comprueba si 'argc' es distinto de 3. Si faltan o sobran argumentos, muestra un mensaje de error explicativo por el canal de error estándar **stderr** y aborta la ejecución con **exit(EXIT_FAILURE)**.
- Si los argumentos son correctos, asigna la dirección de argv[1] a la variable global nombreOrigen y convierte argv[2] a número entero mediante **atoi()** para asignarlo a tamTrozo.

hacha.c

```
void processArgs(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Error. Uso: %s fichero tam_trozo\n",
argv[0]);
        exit(EXIT_FAILURE);
    }
    nombreOrigen = argv[1];
    tamTrozo = atoi(argv[2]);
}
```

2.2.3. Función *calcularTrozos()*

La función **calcularTrozos()** consulta las propiedades del archivo en el sistema de archivos para determinar cuántos fragmentos se generarán:

- Declara una estructura **struct stat propFich** y llama a la función **stat()**, pasándole la ruta del archivo.

- Si **stat()** retorna un valor negativo, indica que el fichero no existe o no se puede acceder a él; muestra un mensaje mediante **perror()** y finaliza con **exit(EXIT_FAILURE)**.
- Si el archivo se obtiene con éxito, calcula la cantidad de trozos dividiendo el tamaño total en bytes entre el tamaño del bloque deseado.
- Si el tamaño total no es un múltiplo exacto, suma una unidad adicional a la variable trozos para guardar los bytes restantes del último bloque.

hacha.c

```
int calcularTrozos(const char *nombre, int tam) {
    struct stat propFich;
    if (stat(nombre, &propFich) < 0) {
        perror("Error. No existe el fichero");
        exit(EXIT_FAILURE);
    }

    int trozos = propFich.st_size / tam;
    if (propFich.st_size % tam != 0) {
        trozos++;
    }
    return trozos;
}
```

2.2.4. Función *divFich()*

Es la función central del programa y se encarga de gestionar las llamadas al sistema para la comunicación mediante tuberías y la creación de procesos:

- Abre el archivo de origen en modo solo lectura mediante la llamada al sistema **open(nombreOrigen, O_RDONLY)**.
- Reserva memoria dinámica con **malloc()** para un buffer temporal de tamaño tamTrozo.
- Mediante un bucle for que itera numTrozos veces:
 - Crea una tubería sin nombre para el canal de comunicación del fragmento actual llamando a **pipe(tubo)**.
 - Genera un proceso hijo mediante **fork()**.
 - En el **proceso padre**, invoca a **ejecPadre()**, pasando el descriptor del archivo origen, el array de la tubería y el buffer.
 - En el **proceso hijo**, cierra el descriptor del archivo origen con **close(fd_origen)** e invoca a **ejecHijo()** para la creación y escritura del fragmento.
- Al finalizar el bucle, el padre cierra el archivo de origen (**close(fd_origen)**) y libera el bloque de memoria dinámica reservado (**free(buffer)**).

- Finalmente, ejecuta un bucle con **wait(NULL)** tantas veces como trozos creados para asegurar la recolección ordenada de todos sus procesos hijos.

Hacha.c

```
void divFich(void) {
    int fd_origen = open(nombreOrigen, O_RDONLY);
    if (fd_origen < 0) {
        perror("Error al abrir el fichero de origen");
        exit(EXIT_FAILURE);
    }

    char *buffer = (char *)malloc(sizeof(char) * tamTrozo);
    if (buffer == NULL) {
        perror("Error de asignación de memoria");
        close(fd_origen);
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < numTrozos; i++) {
        int tubo[2];
        if (pipe(tubo) < 0) {
            perror("Error al crear la tubería");
            break;
        }

        pid_t pid = fork();
        if (pid < 0) {
            perror("Error al crear proceso hijo");
            break;
        }

        if (pid != 0) {
            ejecPadre(fd_origen, tubo, buffer);
        } else {
            close(fd_origen);
            ejecHijo(nombreOrigen, i, tubo, buffer);
        }
    }

    close(fd_origen);
    free(buffer);

    for (int i = 0; i < numTrozos; i++) {
```

```
wait(NULL);  
}  
}
```

2.2.5. Función ejecPadre()

Representa la lógica de lectura y envío, ejecutada por el proceso padre en cada iteración:

- Lee del archivo original un bloque de como máximo **tamTrozo** bytes utilizando la llamada al sistema **read(fd_origen, buffer, tamTrozo)**.
- Si la lectura es satisfactoria (**numLeidos > 0**), escribe dicho bloque en el extremo de escritura de la tubería mediante **write(tubo[1], buffer, numLeidos)**.
- Cierra ambos descriptores de la tubería (**tubo[0]** y **tubo[1]**) con **close()** para notificar el fin del flujo de datos al proceso hijo receptor.

hacha.c

```
void ejecPadre(int fd_origen, int tubo[2], char *buffer) {  
    int numLeidos = read(fd_origen, buffer, tamTrozo);  
    if (numLeidos > 0) {  
        write(tubo[1], buffer, numLeidos);  
    }  
    close(tubo[0]);  
    close(tubo[1]);  
}
```

2.2.6. Función ejecHijo()

Representa la lógica del proceso hijo receptor encargado de persistir el fragmento en disco:

- Llama a **genFrag()** para componer el nombre final del archivo de destino con formato .hXX.
- Crea un nuevo archivo de destino con permisos 0666 haciendo uso de la llamada al sistema **creat(nombreFichero, 0666)**.
- Lee del extremo de lectura de la tubería (**tubo[0]**) los bytes enviados por el padre empleando **read(tubo[0], buffer, tamTrozo)**.
- Si se reciben datos (**numLeidos > 0**), los escribe inmediatamente dentro del archivo fragmento abierto mediante **write(fd_destino, buffer, numLeidos)**.
- Cierra los descriptores de la tubería y del archivo creado, libera el espacio de memoria del buffer con **free(buffer)** y finaliza el proceso de forma limpia con **exit(EXIT_SUCCESS)**.

hacha.c

```
void ejecHijo(const char *nombre, int indice, int tubo[2],
char *buffer) {

char nombreFichero[50];

genFrag(nombreFichero, nombre, indice);

int fd_destino = creat(nombreFichero, 0666);
if (fd_destino < 0) {
    perror("Error al crear fragmento destino");
    close(tubo[0]);
    close(tubo[1]);
    free(buffer);
    exit(EXIT_FAILURE);
}

int numLeidos = read(tubo[0], buffer, tamTrozo);
if (numLeidos > 0) {
    write(fd_destino, buffer, numLeidos);
}

close(tubo[0]);
close(tubo[1]);
close(fd_destino);
free(buffer);
exit(EXIT_SUCCESS);
}
```

2.2.7. Función *genFrag()*

La función **genFrag()** da formato al nombre de salida del fragmento actual manteniendo el nombre base del archivo de entrada:

- Para índices menores que 10, concatena la extensión .h0X anteponiendo un cero mediante **sprintf(outNombre, "%s.h0%d", base, indice)**.
- Para índices mayores o iguales que 10, concatena la extensión .hX sin cero inicial usando **sprintf(outNombre, "%s.h%d", base, indice)**.

hacha.c

```
void genFrag(char *outNombre, const char *base, int indice){

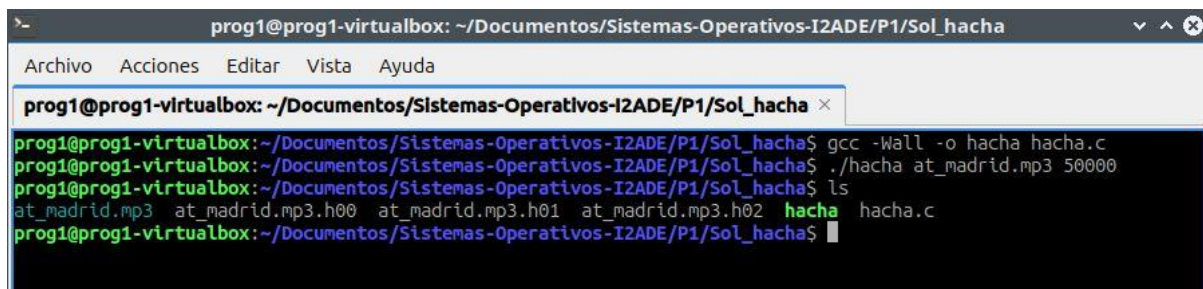
    if (indice < 10) {

        sprintf(outNombre, "%s.h0%d", base, indice);

    }
```

```
}  
  
else {  
  
    sprintf(outNombre, "%s.h%d", base, indice);  
  
}  
  
}
```

2.3. Salida esperada



The screenshot shows a terminal window titled "prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hacha". The terminal displays the following commands and output:

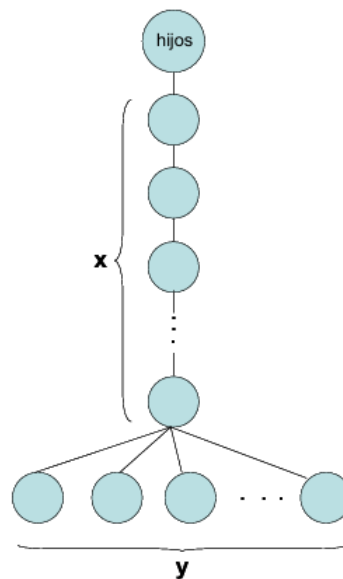
```
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hacha$ gcc -Wall -o hacha hacha.c  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hacha$ ./hacha at_madrid.mp3 50000  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hacha$ ls  
at_madrid.mp3  at_madrid.mp3.h00  at_madrid.mp3.h01  at_madrid.mp3.h02  hacha  hacha.c  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hacha$
```

Ejercicio 3:

3.1. Enunciado del ejercicio

Diseña un programa llamado `hijos.c` que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros

`$hijos x y`



El proceso **hijos** debe mostrar el siguiente mensaje:

“Soy el superpadre (pid) : mis hijos finales son: `pidx1`, `pidx2`, `pidx3`, ..., `pidxy`”

Los procesos de la parte baja deben mostrar el siguiente mensaje:

“Soy el subhijo `pidx1`, mi padres son: `pid1`, `pid2`, ..., `pidx`”

3.2. Funciones

3.2.1. Variables globales

Dado que múltiples procesos necesitan acceder y modificar información simultáneamente en distintos puntos de ejecución, se hace uso de variables globales para la gestión de dimensiones y configuración de memoria IPC (Inter-Process Communication):

- **x, y:** variables de tipo `int` que representan la cantidad de procesos verticales (profundidad) y horizontales (anchura) que tendrá el árbol, respectivamente.
- **idMemoriaX, idMemoriaY:** variables de tipo `integer` que almacenarán los identificadores de los segmentos de memoria compartida devueltos por el sistema.

- **pidsX, pidsY**: punteros de tipo integer donde se vincularán las direcciones de los segmentos de memoria compartida para almacenar los PIDs de los procesos.
- **pidSuperPadre**: variable de tipo `pid_t` que almacenará el PID del proceso raíz o principal para poder imprimirlo posteriormente en la descendencia.

```
hijos.c
```

```
int x, y;

int idMemoriaX, idMemoriaY;

int *pidsX, *pidsY; pid_t pidSuperPadre;
```

3.2.2. Función *main()*

La función **main()** organiza la secuencia completa del programa y actúa como el "super padre" de toda la jerarquía. Lo primero que hace es validar y procesar los argumentos mediante **processArgs()**. A continuación, almacena su PID en `pidSuperPadre` haciendo uso de **getpid()** y llama a **confMem()** para solicitar y vincular los segmentos de memoria compartida.

Posteriormente, invoca a **cadena_vertical()** para arrancar la creación de procesos en profundidad. Una vez que todos los procesos descendientes (verticales y horizontales) finalizan, el flujo principal retorna al super padre, que llama a **printSuperPadre()** para imprimir el listado de PIDs finales recolectados. Por último, invoca a **cleanMem()** para destruir definitivamente la memoria compartida y retorna 0.

```
hijos.c
```

```
int main(int argc, char *argv[]) {
    processArgs(argc, argv);

    pidSuperPadre = getpid();
    confMem();

    cadena_vertical();

    printSuperPadre();

    cleanMem();

    return 0;}
```

3.2.3. Gestión de Memoria Compartida (*confMem* y *cleanMem*)

El intercambio de información entre procesos independientes requiere reservar memoria del sistema:

- **confMem()**: Emplea **shmget(IPC_PRIVATE, ... , IPC_CREAT | 0666)** para crear dos segmentos del tamaño necesario. A continuación, los vincula al espacio de direcciones del proceso actual usando **shmat()**. Cualquier error aborta el programa.
- **cleanMem()**: Se ejecuta en el super padre tras finalizar la actividad. Primero desvincula las variables con **shmdt()** y luego destruye definitivamente el segmento de memoria IPC en el sistema invocando **shmctl(..., IPC_RMID, NULL)**.

```
hijos.c
void confMem(void) {
    idMemoriaX = shmget(IPC_PRIVATE, sizeof(int) * x,
IPC_CREAT | 0666);
    if (idMemoriaX < 0) {
        perror("Error en shmget X");
        exit(EXIT_FAILURE);
    }
    pidsX = (int *)shmat(idMemoriaX, 0, 0);
}
```

3.2.4. Función *cadena_vertical()* y *printInfo*

Esta función construye el tramo descendente del árbol mediante iteración y bifurcación:

- Un bucle iterará x veces ejecutando la llamada **fork()** en cada ciclo.
- **Proceso Padre**: Ejecuta **wait(NULL)** para esperar la finalización de su hijo y realiza un break para abandonar el bucle, asegurando la destrucción ordenada de abajo hacia arriba.
- **Proceso Hijo**: Almacena su PID obtenido con **getpid()** en la posición correspondiente del array compartido pidsX e invoca a **printInfo()** para listar a todos sus antecesores (recorriendo el propio array pidsX).
- Al alcanzar el último eslabón vertical ($i == x + 1$), el último proceso hijo invoca **hoja_horizontal()** para bifurcar la parte final del árbol. Tras ello, todos los hijos desvinculan su memoria con **shmdt()** y mueren limpiamente mediante **exit(0)**.

```
hijos.c
void cadena_vertical(void) {
    int i;
    pid_t pid;
```

```

    for (i = 1; i <= x; i++) {
        pid = fork();
        if (pid != 0) {
            wait(NULL);
            break;
        } else {
            pidsX[i - 1] = getpid();
            printInfo(i);
        }
    }
    if (i != 1) {
        if (i == x + 1) {
            hoja_horizontal();
        }
        shmdt(pidsX);
        shmdt(pidsY);
        exit(0);
    }
}

```

3.2.5. Funciones horizontales (*hoja_horizontal* y *ejec_horizontal*)

El último proceso de la cadena vertical ejecuta **hoja_horizontal()** para crear concurrentemente los y nodos finales:

- Se lanza un bucle for que invoca a **fork()** y veces de forma paralela (sin wait intermedios).
- Cada proceso hijo resultante llama a **ejec_horizontal(i - 1)**.
- El proceso que iteraba (padre de este nivel) ejecuta posteriormente un bucle adicional con **wait(NULL)** para recolectar a todos los hijos generados concurrentemente.

Dentro de **ejec_horizontal()**, cada nodo hoja inscribe su identificador en el segmento **pidsY[indice]**. Para garantizar que permanezcan vivos lo suficiente como para verificar el árbol en el sistema, configuran una alarma de 10 segundos (**alarm(10)**) y entran en bloqueo con **pause()**. Tras recibir la señal SIGALRM, efectúan el desanclaje de memoria compartida y ejecutan **exit(0)**.

```

hijos.c
void ejec_horizontal(int indice) {
    pidsY[indice] = getpid();

    signal(SIGALRM, controlAlarma);
    alarm(10);
    pause();
}

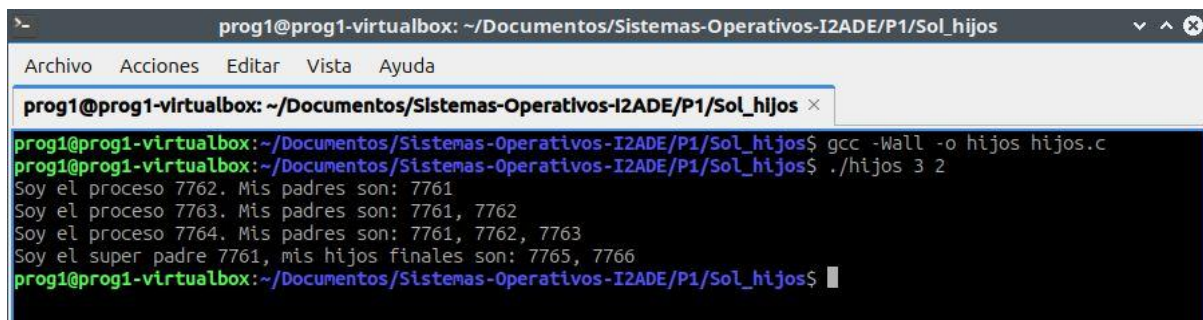
```

```
shmdt(pidsX);  
shmdt(pidsY);  
exit(0);  
}
```

3.2.6. Función *printSuperPadre()*

Se ejecuta exclusivamente en el proceso inicial tras finalizar las esperas. Recorre secuencialmente la memoria compartida pidsY (que ha sido rellenada en paralelo por los procesos hojas distantes) e imprime los PIDs correspondientes, demostrando el éxito en la comunicación inter-procesos a través del árbol genealógico completo.

3.3. Salida esperada



```
prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hijos  
Archivo Acciones Editar Vista Ayuda  
prog1@prog1-virtualbox: ~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hijos ×  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hijos$ gcc -Wall -o hijos hijos.c  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hijos$ ./hijos 3 2  
Soy el proceso 7762. Mis padres son: 7761  
Soy el proceso 7763. Mis padres son: 7761, 7762  
Soy el proceso 7764. Mis padres son: 7761, 7762, 7763  
Soy el super padre 7761, mis hijos finales son: 7765, 7766  
prog1@prog1-virtualbox:~/Documentos/Sistemas-Operativos-I2ADE/P1/Sol_hijos$
```

GITHUB

<https://github.com/Sergioooo7/Sistemas-Operativos-I2ADE.git>